

Activity A1 – From Idea to Project Specification

Title: Define Your Mini-Project Clearly

Goal

Turn a rough idea into a precise, buildable project description.

Instructions

Pick one idea (examples: simple hospital appointment system, dataset analyzer, quiz app, log analyzer, file organizer).

Part A – One-page project brief

1. Project title:

Agri Rover Environment monitor

2. Problem you want to solve (2–3 short sentences):

Real-time analysis of localised soil data
(pH, Temp, humidity) to optimise crop health

3. Target users (who will actually use this?):

Agri researchers / scientists / Farmers

4. Inputs (what data / actions will the program receive?):

- Data on pH, Temp, humidity
- GPS coordinates

5. Outputs (what will the user see / get?):

- A time stamped log file & table showing current soil
- activities to do further to amend any issues. ^{health}

6. Constraints or assumptions (keep it realistic):

- minimal value as info is variable
- Reading of all three sensory values into a file

Part B – Core features vs “nice-to-have”

1. List minimum features needed for a working version (MVP):

- data from sensors
- data logs on device
- display of data & the working condition of sensors

2. List optional features (only if time allows):

- controlling the rover manually
- Alarm system in the rover.

3. Final question:

- If you had only 1 week, which ONE feature would you absolutely keep? Why?

Sensor data logs as we can use that information
to address the condition of soil & take necessary action

Activity A2 – System Decomposition (Modules & Responsibilities)

Title: Break Project into Modules

Goal

Learn to split a project into logical modules and functions.

Instructions

Use your project from Activity A1.

Part A – High-level modules

1. List 3–6 modules (e.g., data_input, processing, reporting, ui):

- Module 1: sensor-driver
- Module 2: data-processor
- Module 3: storage-manager
- Module 4: alert-handler

2. For each module, write its main responsibility in one sentence:

- Module 1 does: manage info from sensors to device & vice versa
- Module 2 does: convert data signal to physical units
- Module 3 does: handle the csv file to open, write & close the file
- Module 4 does: check the live value against pre-set value as threshold & triggers the alarm

Part B – Functions inside modules

For one key module (choose the most important):

1. Module name: sensor-driver

2. Functions planned:

- Function 1 name: read-soil-ph()
 - Input(s): analog pin number (int)
 - Output: voltage reading (float)
- Function 2 name: read-dht-data()
 - Input(s): digital pin no (int), sensor ^{type} ~~type~~ (str)
 - Output: Tuple containing data (temp,
- Function 3 name: initialise-all-sensors()
 - Input(s): —
 - Output: Boolean (True/False). if ready or not

3. Draw or briefly describe how data flows between these functions:

data flow begin with "initialise-all-sensors" to ensure hardware ready.
then loop reads for "read-soil-ph" & "read-dht-data".
these raw data pass through "data-processor" for
calibration before sending to "storage-manager" for
data logs on device

Activity A3 – Designing Data Structures

Title: Choosing the Right Data Structures

Goal

Practice selecting lists, dicts, sets, tuples, or custom classes.

Instructions

Consider your project's core data (e.g., patient records, quiz questions, log entries, transactions).

Part A – Describe one key entity

1. Entity name (e.g., "Appointment", "Record", "Transaction"):

Soil Reading

2. For this entity, list required fields:

- Field 1: time stamp (type: int/str/float/...)
- Field 2: ph-level (type: float)
- Field 3: temp (type: float)
- Field 4: humidity (type: float)

Part B – Pick implementation

Choose one representation and justify:

1. Option A – Dictionary:

python

item = {

"field1": _____,

"field2": _____,

...

}

reading = {

"timestamp": 2026-02-16 10:00:00,

"ph_level": 6.5,

"temp": 24.3,

"humidity": 45.2

}

2. Option B – Class:

python

class Item:

def __init__(self, field1, field2, ...):

self.field1 = field1

self.field2 = field2

...

3. Which option will you use, and why?

option A, as it is easily accessible to read values & display it.

4. Write a short example of creating and using this structure:

python

Example creation and simple usage

Activity A4 – Planning Tests Before Coding

Title: Think in Test Cases (TDD Lite)

Goal

Build habit of designing tests before writing code.

Instructions

Choose one non-trivial function from your project (e.g., fee calculation, filter logic, scoring rule).

1. Function name: caliberate_ph_reading
2. In simple words, what should this function do?

3. List at least 5 test cases (inputs and expected outputs):

Test #	Input(s) <i>voltage</i>	Expected output <i>pH</i>	Logic <i></i>
1	2.5	7.0	neutral point
2	1.5	4.0	Acid threshold test value
3	3.5	10.0	Alkaline threshold test value
4	-1	error	physical impossibility
5	5	14.0	max value.

4. Now write a small test harness:

python

def my_func(...):

TODO: implement

pass

Tests

print("Test 1:", my_func(...), "expected:", ...)

print("Test 2:", my_func(...), "expected:", ...)

5. Reflection:

- Did writing tests first change how you thought about the function? How?

Yes, it helped me understand, what inputs are put & the way it is converted & displayed.

Activity A5 – Error Handling & Robustness in a Project

Title: Make Your Project Hard to Break

Goal

Identify and protect weak points with validation and exception handling.

Instructions

Look at your project idea and identify three likely failure points (e.g., wrong file path, invalid input, network issue).

1. Failure Point 1: Sensor disconnection

- How to detect it? error when python script communicate with sensor.
- How to handle it politely for the user? log that specific sensor failure to user.

2. Failure Point 2: Invalid data / out-of-range reading

- Detect: using conditional logic to check if value is impossible
- Handle: discard those data as anomaly

3. Failure Point 3: Storage full

- Detect: if error while storing data
- Handle: trigger on device to inform data not being stored.

Part B – Add try/except around one risky area

Example skeleton:

python

```
file_path = input("Enter file path: ")
```

```
try:
```

```
    with open(file_path, "r") as f:
```

```
        data = f.read()
```

```
    print("File loaded successfully.")
```

```
except FileNotFoundError:
```

```
    print("File not found. Please check the path.")
```

```
except PermissionError:
```

```
    print("Permission denied. Try a different file.")
```

Adapt this pattern to your project and paste your modified code below:

python

Your robust code:

```
def get_sensor_data():
```

```
    try:
```

```
        ph_value = sensor_lib.read_ph()
        print("sensor reading successful")
```

```
    except OSError:
```

```
        print("Error: no connection to pH sensor")
        ph_value = None
```

```
    except Exception as e:
```

```
        print(f"An unexpected error occurred: {e}")
```

```
    return ph_value
```

Activity A6 – Refactoring for Clarity

Title: Improve Existing Code (Same Behaviour, Better Design)

Goal

Practice cleaning up messy, working code into clear, modular code.

Instructions

Take one working script (from your own work or given by teacher).

It should be correct but long, repetitive, or hard to read.

1. Paste or summarize original code name / purpose:

move_data - v1.py

2. Identify at least three problems (long function, repetition, confusing names, etc.):

- Problem 1: long code
- Problem 2: Repetitive code
- Problem 3: confusing variable name

3. Plan refactoring steps (high level):

- Step 1: extract sensor reading logic into individual
- Step 2: save_to_csv() function to handle all file operation
- Step 3: Rename variable to agri terms.

4. After refactoring, write or paste your improved version here:

python

Refactored code:

5. Reflection:

- What became easier to understand after refactoring?

Activity A7 – Building a Command-Line Interface (CLI)

Title: Make Your Project Usable from Terminal

Goal

Learn to take arguments and options from the command line.

Instructions

Turn some functionality of your project into a small CLI tool.

1. Define a simple CLI command:

Example:

- `python student_tool.py add --name Alice --score 85`

2. What should the tool do in ONE sentence?

let farmers input their soil details & requirements.

3. Sketch CLI arguments:

Argument / Option	Meaning
<i>PH</i>	<i>PH of soil</i>
<i>voltage</i>	<i>EC of soil</i>
<i>sensors</i>	<i>sensor data</i>

4. Implement with argparse (skeleton):

```
python
```

```
import argparse
```

```
parser = argparse.ArgumentParser(description="Describe your tool here")
```

```
parser.add_argument("--name", type=str, help="Student name")
```

```
parser.add_argument("--score", type=int, help="Student score")
```

```
args = parser.parse_args()
```

```
# Use args.name, args.score here
```

```
print("Name:", args.name)
```

```
print("Score:", args.score)
```

5. Adapt the skeleton to your project and note example commands at the bottom of your file.

Activity A8 – Mini Capstone: End-to-End Project Plan & Review

Title: From Zero to Demo

Goal

Produce a realistic plan and self-review for your project.

Part A – Milestones and Timeline

1. List 4–6 milestones (each should be testable):

- Milestone 1: Hardware driver (analog to digital)
- Milestone 2: Data logic (code calibration)
- Milestone 3: Robustness (error handling)
- Milestone 4: Field interface (health check line status)
- Milestone 5: user interface (remote handling)

2. For each milestone, define:

Milestone	Concrete Deliverable	Due Date
1	Analog to Digital Convertable	week 1
2	convert Formula to Readable	week 2
4	Real to give quick health update	week 3.

Part B – Risk & Mitigation

1. Identify 2–3 biggest risks (technical or time):

- Risk 1: Sensor degradation
 - Plan B if this fails: back up sensor
- Risk 2: Power fluctuation
 - Plan B if this fails: battery
- Risk 3: Environment hazard
 - Plan B if this fails: protective gear

Part C – Final reflection after project

(Do this at the end.)

1. When I got stuck, what strategies actually helped me get unstuck?

Break complex problem to simple

2. One thing I now believe about myself as a problem-solver:

Ability to code

3. If I start a new project tomorrow, what will I do differently from day 1?

I'll start with a plan & execute it

Advanced Problem-Solving Lab – High-IQ Track

Activity H1 – Solve the “Hidden Problem” Behind the Stated Problem

Title: From Surface Problem to Real Problem

Goal

Train yourself to see the *real* problem, not just the stated one.

Instructions

You are given a “client statement”:

“Our exam system is too slow. We want a Python script to speed it up.”

1. Identify at least 3 possible *real* problems this statement might hide.

- Hidden problem 1: sensor obstruction
- Hidden problem 2: signal loss
- Hidden problem 3: soil terrain

2. For each hidden problem, state:

- What data would you need to confirm it?

- Problem 1: mud / damage to sensor probe
- Problem 2: weak signal strength
- Problem 3: compactness of soil hinders reading

3. Choose ONE hidden problem and re-frame it as a clear technical problem statement (2–3 lines):

New problem statement:

system using delta is getting delayed due to weak signal & response are varied & not precise

4. Design a very small Python experiment that could validate or reject your reframed problem (pseudo-code is fine):

python

Experiment idea (what will you measure, on what data, how?)

```
import time
signals = [20, 40, 60, 80]
def measure_response(signal):
    delay = (100 - signal) / 40
    time.sleep(delay)
    return delay
for s in signals:
    start = time.time()
    response = measure_response(s)
end = time.time()
print("Signal:" + s + " Response time:" + end - start)
```

Activity H2 – Multiple Competing Solutions

Title: Design 3 Solutions Before Coding 1

Goal

Avoid fixation. Explore at least three distinct solution strategies.

Instructions

Problem:

"You receive a stream of log lines (text). You must detect 'suspicious' activity in real time."

1. Give three different solution ideas, each using a distinct main technique or data structure.

- Approach A (e.g., rule-based / regex / thresholds):

detect weak signal using fixed threshold

- Approach B (e.g., statistical / sliding window / counters):

monitor signal strength over time using sliding window

- Approach C (e.g., simple ML / anomaly detection concept):

use anomaly detection to identify unusual pattern

2. For each approach, list pros and cons (at least one each).

- Approach A – Pros: simple Cons: not flexible
- Approach B – Pros: trend Cons: tuning
- Approach C – Pros: detect Cons: complex
subwindow

3. Choose one approach to implement first and justify:

I choose approach B because

it can track changes in signal over time & detect drops faster!

4. Sketch high-level Python structure (modules/classes, not full code):

python

Modules / classes / key functions list

```
class SignalMonitor:
```

```
    def __init__(self):  
        pass
```

```
    def add_signal(self, value):  
        pass
```

```
    def detect_drop(self):  
        pass
```

```
def main():  
    pass
```

Activity H3 – Self-Constrained Challenge: “Elegant Code Only”

Title: Solve with Elegance (Clarity, Minimalism, Correctness)

Goal

Produce code that is not only correct, but also minimal and beautiful.

Problem

Implement a mini “pipeline” for processing a list of tasks:

- Each task has an ID, description, and status ("todo", "doing", "done").
- You must:
 - Load tasks from a JSON file.
 - Filter by status.
 - Sort by ID.
 - Print a clean summary.

Constraints (you impose on yourself)

1. Maximum 25 lines of *real* Python (excluding blanks and comments).
2. No function longer than 7 lines.
3. Names must be meaningful and short.

Steps

1. Design your data model in 2–3 lines:

Task = {id, description, status}

2. Decide at least one thing you will *not* do to keep it minimal:

-not use complex class

3. Write your code here (respecting constraints):

python

Elegant pipeline solution

4. Reflection:

- What did you have to sacrifice (if anything) to keep elegance?

I avoided adding extra features to keep it short.

import json

def load(file):

with open(file) as f:

return json.load(f)

def filter_task(task, status):

return [t for t in tasks if t['status'] == ~~task~~ status]

Activity H4 – Complexity and Scaling Thought Experiment

Title: How Will Your Solution Break at Scale?

Goal

Develop realistic intuition about performance and scaling.

Instructions

Take ANY non-trivial function or algorithm you have recently written (search, summarization, scoring, etc.).

1. Describe what it does, in *one line*:

Filter & sort task data

2. Estimate its time complexity in Big-O notation, and justify in 1–2 lines:

- Big-O: $O(n \log n)$
- Because: sorting takes most time

3. Scaling scenarios – imagine input sizes:

- $n = 10^3$ (1,000)
- $n = 10^6$ (1,000,000)
- $n = 10^9$ (1,000,000,000)

For each, predict: will it still be usable (Y/N) and why?

n	Usable (Y/N)	Reason (1 short sentence)
10^3	yes	fast
10^6	yes	manageable
10^9	no	Too slow & high memory

4. Suggest one change to improve scalability and briefly describe how it helps:

use database/indexing to avoid full sorting.

Activity H5 – Adversarial Thinking: Break Your Own Design

Title: Attack Your Own Program

Goal

Develop robustness and defensive mindset.

Instructions

Choose one of your own projects or mini-tools (file reader, API client, CLI, etc.).

1. Write a short description of what it is supposed to do:

Program that reads sensor data & process it

2. Now wear the hat of an *attacker* or malicious user.

List 5 ways to misuse / break it:

- Attack 1: very large data input
- Attack 2: wrong data format
- Attack 3: mixing value
- Attack 4: weak signal value
- Attack 5: corrupted file

3. For each attack, suggest one defence in plain language:

- Defence for attack 1: limit input size
- Defence for attack 2: validate format
- Defence for attack 3: Handle mixing data
- Defence for attack 4: set threshold
- Defence for attack 5: check file integrity.

4. Implement at least one of these defences in your code and paste the updated snippet:

python

Hardened code snippet:

```
def process(data):  
    if not data:  
        return "No data"  
    for d in data:  
        if d < 0:  
            return "invalid"  
    return "processed"
```

Activity H6 – Cross-Domain Transfer

Title: Apply Known Pattern to a New Domain

Goal

Practice lifting a known idea/pattern from one domain to a different one.

Instructions

Pick a pattern you already know well, e.g.:

- Sliding window
- Two-pointer technique
- Dynamic programming table
- Producer-consumer
- Map-reduce style
- Pipeline of filters

1. Name of pattern: Sliding window
2. Original domain where you learned it (e.g., arrays, strings):

Arrays

3. New domain to apply it in (e.g., logs, time series, learning activities, medical data):

signal monitoring

4. Describe how you would adapt the pattern to this new domain, in 5–8 lines (can include pseudo-code):

using sliding window to track signal values
remove old value & check average, detect sudden
drop in signal & raise alert.

5. Optional: implement a small prototype showing this transfer:

python

Pattern transfer prototype

```
from collections import deque  
window = deque()
```

```
def add-signal(val):  
    window.append(val)  
    if len(window) > 5:  
        window.popleft()
```

```
    if sum(window)/len(window) < 30:  
        print("weak signal")
```


Activity H7 – Meta-Cognition: Your Personal Problem-Solving Manual

Title: Write Your Own "How I Solve Problems" Guide

Goal

Turn your tacit skills into an explicit, reusable system.

Instructions

1. List the first three things you usually do when you get a new problem (be honest):

- Step 1: Read question
- Step 2: start coding
- Step 3: Fix errors.

2. Now, design your ideal 6–8 step process (what a "future you" would always do):

1. understand problem
2. identify issue
3. plan solution
4. compare options
5. choose best
6. design & execute
7. (optional) _____
8. (optional) _____

3. For each step, write one question you will ask yourself:

- Step 1 question: what is real problem
- Step 2 question: why is it happening
- Step 3 question: which are options
- Step 4 question: how it is best
- Step 5 question: tools which
- Step 6 question: how to test

4. Final instruction to yourself (one sentence):

"When I feel stuck, I will rethink before coding